
date = '2026-06-11T19:05:19+08:00'

draft = true

title = 'mall-swarm介绍和部署'

image = '项目架构.png'

mall-swarm 阿里云 ACK 部署完整全流程实战

项目：基于 Spring Cloud Alibaba 的微服务商城系统（mall-swarm）在阿里云托管版 Kubernetes（ACK）集群上的全流程部署

目标：将完整的 mall-swarm（含前端）部署到阿里云 ACK 集群，使用个人容器镜像仓库，解决国内网络限制与配置适配问题，实现公网可访问。

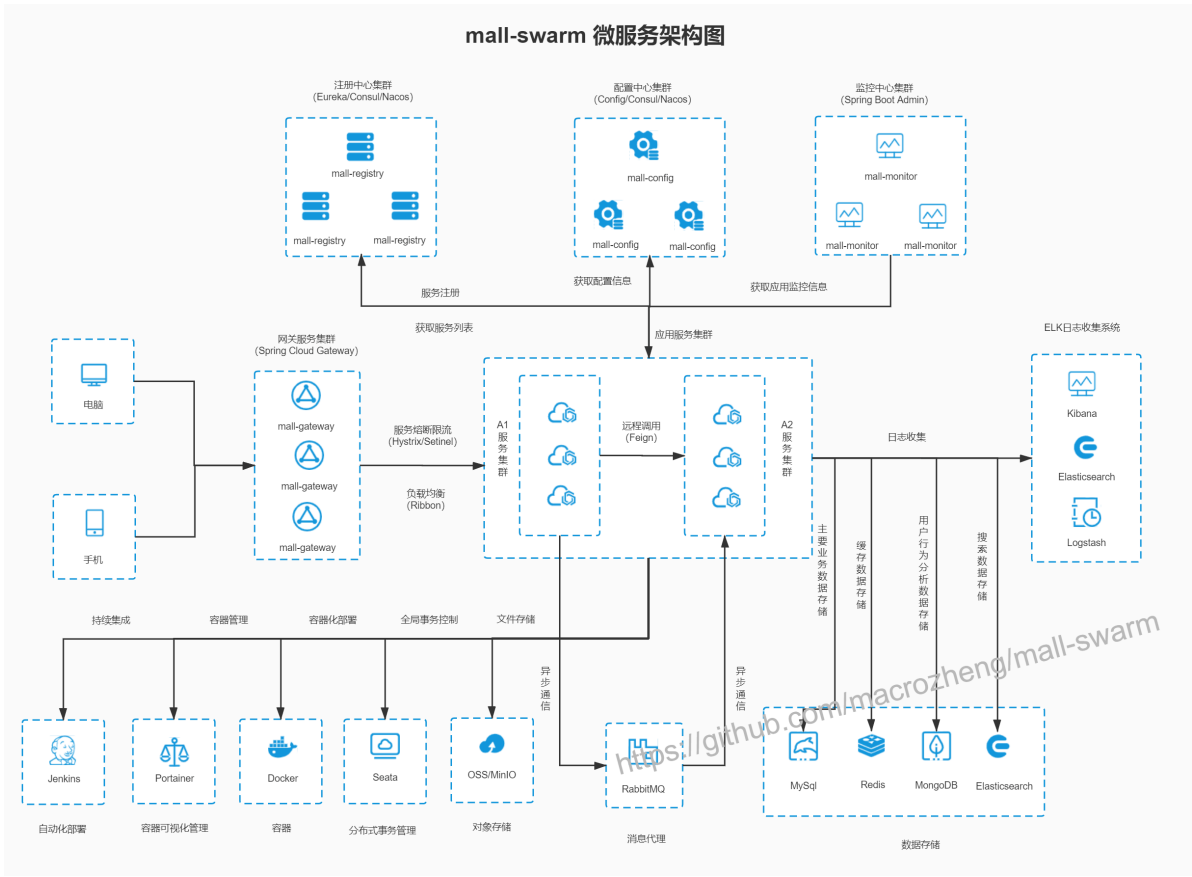
一、项目介绍

mall-swarm 是一套微服务商城系统，采用了 Spring Cloud Alibaba、Spring Boot 3.5、Sa-Token、MyBatis、Elasticsearch、Docker、Kubernetes等核心技术，同时提供了基于Vue的管理后台方便快速搭建系统。mall-swarm 在电商业务的基础集成了注册中心、配置中心、监控中心、网关等系统功能。

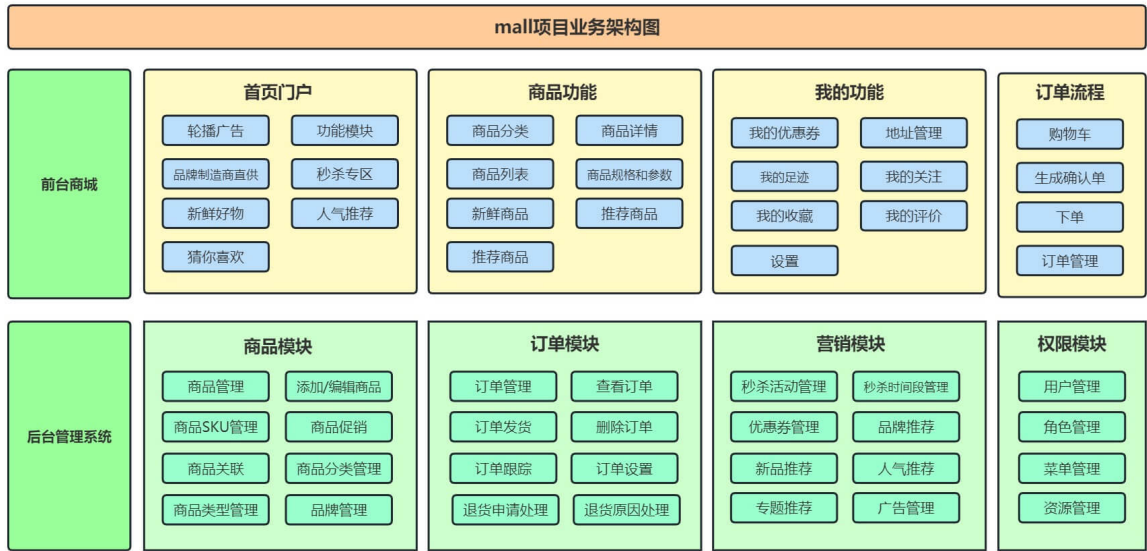
官方资源：

- GitHub 官方仓库：<https://github.com/macrozheng/mall-swarm>
- Gitee 官方仓库：<https://gitee.com/macrozheng/mall-swarm>
- 官方全套文档站点：<https://www.macrozheng.com>

1. 架构与组织结构



组织架构:



系统组织结构大概如下:

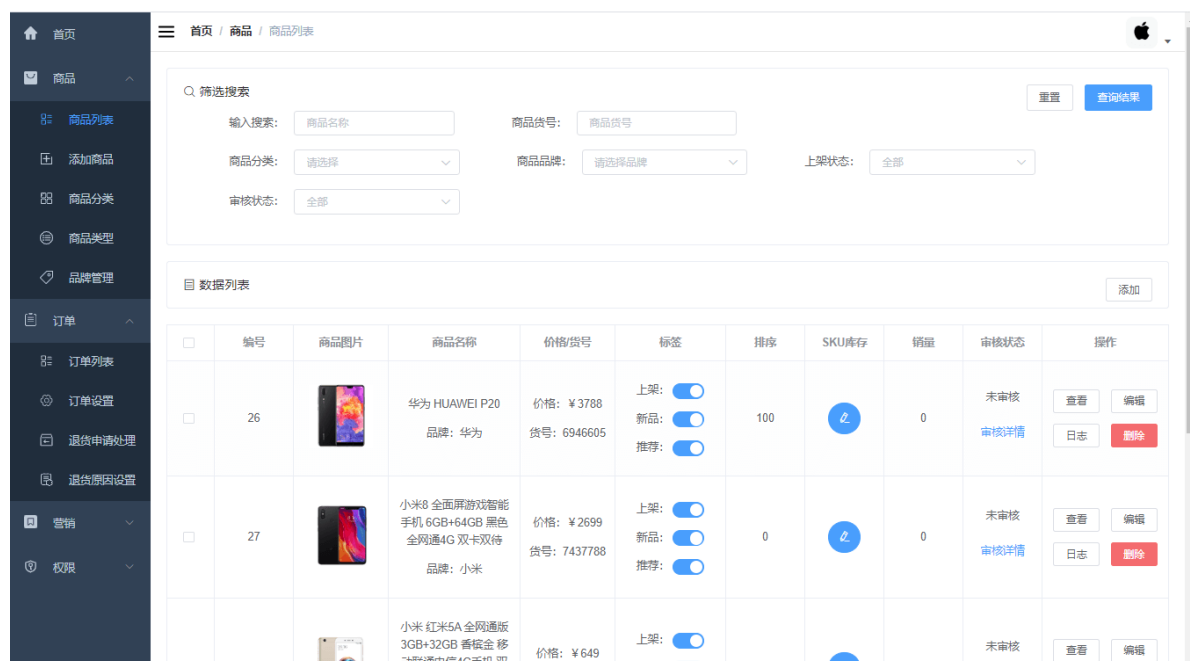
```
mall
├─ mall-common -- 工具类及通用代码模块
├─ mall-mbg -- MyBatisGenerator生成的数据库操作代码模块
├─ mall-auth -- 基于Spring Security OAuth2的统一的认证中心
├─ mall-gateway -- 基于Spring Cloud Gateway的微服务API网关服务
├─ mall-monitor -- 基于Spring Boot Admin的微服务监控中心
├─ mall-admin -- 后台管理系统服务
├─ mall-search -- 基于Elasticsearch的商品搜索系统服务
├─ mall-portal -- 移动端商城系统服务
├─ mall-demo -- 微服务远程调用测试服务
└─ config -- 配置中心存储的配置
```

2. 系统展示

后台管理系统

前端项目 `mall-admin-web` 地址: <https://github.com/macrozheng/mall-admin-web>

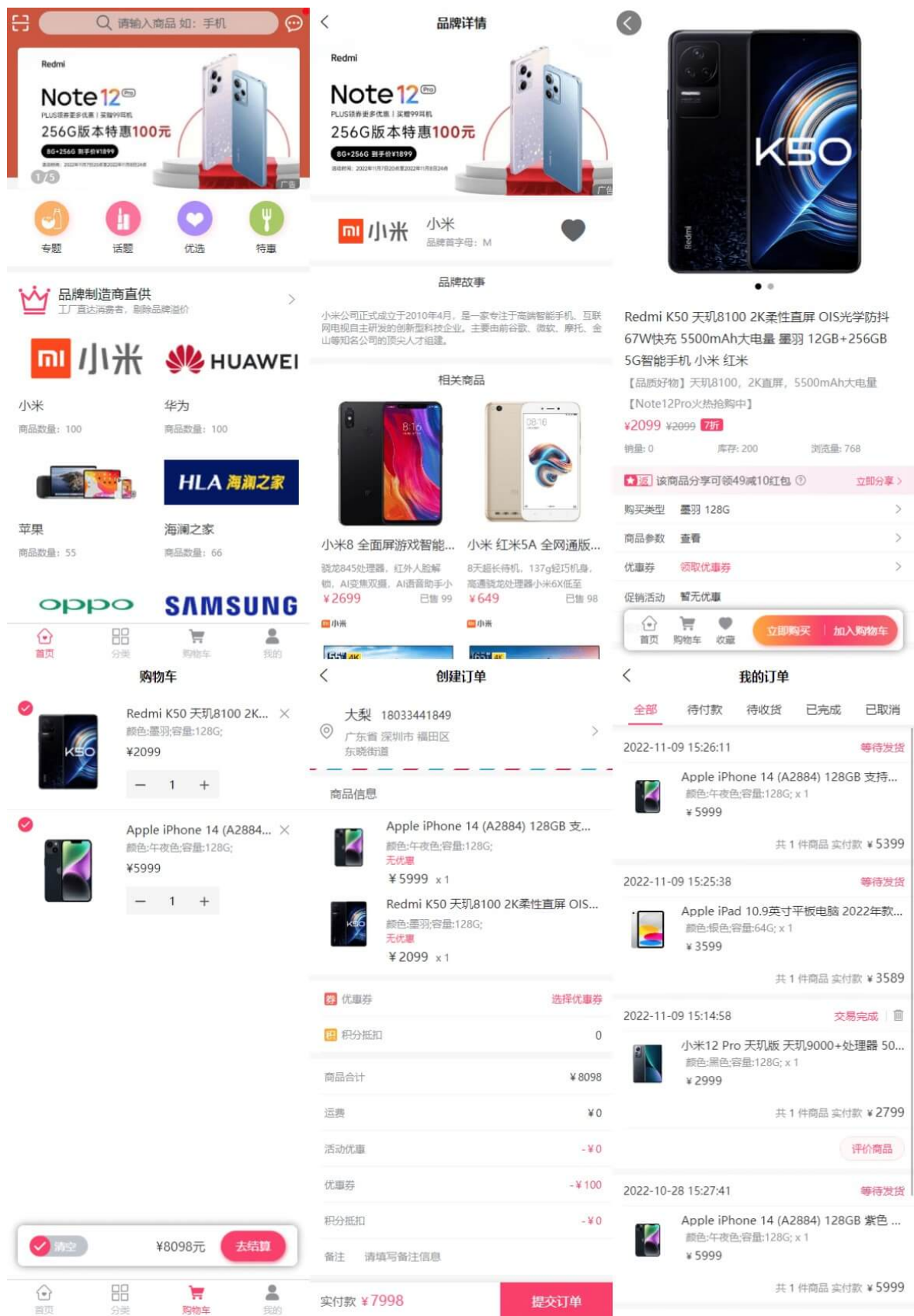
项目演示地址: <https://www.macrozheng.com/admin/index.html>



前台中城系统

前端项目 `mall-app-web` 地址: <https://github.com/macrozheng/mall-app-web>

项目演示地址 (将浏览器切换为手机模式效果更佳): <https://www.macrozheng.com/app/>



3. 技术栈说明

后端技术:

技术	说明	官网
Spring Cloud	微服务框架	https://spring.io/projects/spring-cloud
Spring Cloud Alibaba	微服务框架	https://github.com/alibaba/spring-cloud-alibaba
Spring Boot	容器+MVC框架	https://spring.io/projects/spring-boot
Sa-Token	认证和授权框架	https://github.com/dromara/Sa-Token
MyBatis	ORM框架	http://www.mybatis.org/mybatis-3/zh/index.html
MyBatisGenerator	数据层代码生成	http://www.mybatis.org/generator/index.html
PageHelper	MyBatis物理分页插件	http://git.oschina.net/free/Mybatis_PageHelper
Knife4j	文档生产工具	https://github.com/xiaoymin/swagger-bootstrap-ui
Elasticsearch	搜索引擎	https://github.com/elastic/elasticsearch
RabbitMQ	消息队列	https://www.rabbitmq.com/
Redis	分布式缓存	https://redis.io/
MongoDb	NoSql数据库	https://www.mongodb.com/
Docker	应用容器引擎	https://www.docker.com/
Druid	数据库连接池	https://github.com/alibaba/druid
OSS	对象存储	https://github.com/aliyun/aliyun-oss-java-sdk
MinIO	对象存储	https://github.com/minio/minio
LogStash	日志收集	https://github.com/logstash/logstash-logback-encoder
Lombok	简化对象封装工具	https://github.com/rzwitserloot/lombok
Seata	全局事务管理框架	https://github.com/seata/seata
Portainer	可视化Docker容器管理	https://github.com/portainer/portainer
Jenkins	自动化部署工具	https://github.com/jenkinsci/jenkins
Kubernetes	应用容器管理平台	https://kubernetes.io/

前端技术（后台管理）：

技术	说明	官网
Vue	前端框架	https://vuejs.org/
Vue-router	路由框架	https://router.vuejs.org/
Vuex	全局状态管理框架	https://vuex.vuejs.org/
Element	前端UI框架	https://element.eleme.io/
Axios	前端HTTP框架	https://github.com/axios/axios
v-charts	基于Echarts的图表框架	https://v-charts.js.org/

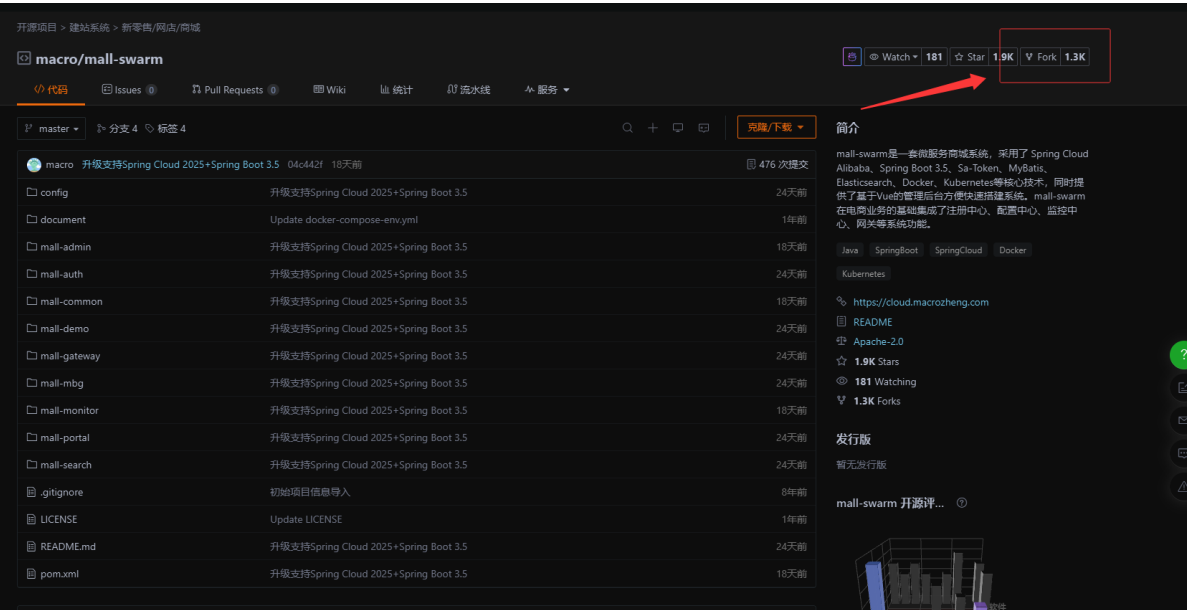
移动端技术（前台中城）：

技术	说明	官网
Vue	核心前端框架	https://vuejs.org
Vuex	全局状态管理框架	https://vuex.vuejs.org
uni-app	移动端前端框架	https://uniapp.dcloud.io
mix-mall	电商项目模板	https://ext.dcloud.net.cn/plugin?id=200
luch-request	HTTP请求框架	https://github.com/lei-mu/luch-request

二、本地环境搭建与后端准备

为了节省资源采用了部署跟准备分开，提前在本地 docker 上构建相关镜像并且推送到镜像仓库里面。我们需要准备 git、docker、docker-compose、kubectrl、阿里云 CLI、openjdk(至少17)、maven。

本次代码仓库使用的是 gitee，相关代码存放到 gitee 上。我们将官方地址 Fork 到自己的镜像仓库里面：<https://gitee.com/erha6657/mall-swarm>



采用的是 centos7 系统 IP：192.168.10.44。

1. 安装基础环境

安装必要的依赖并配置阿里云 Docker YUM 源，和安装 docker-compose:

```
yum install -y yum-utils device-mapper-persistent-data lvm2

yum-config-manager --add-repo https://mirrors.aliyun.com/docker-
ce/linux/centos/docker-ce.repo
yum install -y docker-ce docker-ce-cli containerd.io

mkdir -p /etc/docker

cat > /etc/docker/daemon.json <<EOF
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://dockerhub.icu",
    "https://registry.cn-hangzhou.aliyuncs.com"
  ],
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m",
    "max-file": "3"
  }
}
EOF

systemctl daemon-reload
systemctl enable docker
systemctl start docker

yum install -y docker-compose-plugin
```

注: 本次使用的 Docker version 为 26.1.4, Docker Compose version 为 v2.27.1。

安装阿里云 CLI (用于登录镜像仓库) :

```
/bin/bash -c "$(curl -fsSL https://aliyuncli.alicdn.com/install.sh)"

aliyun version
# 3.3.22
```

安装 kubectl:

```
# 使用 IPv4 下载官方 1.36.1 版本的二进制文件
curl -4 -LO "https://dl.k8s.io/release/v1.36.1/bin/linux/amd64/kubectl"
# 给文件赋予可执行权限
chmod +x ./kubectl

# 移动到系统的全局可执行目录中
mv ./kubectl /usr/local/bin/kubectl

kubectl version --client --output=yaml
```

安装 Git:

```
yum install git-all -y
```

2. 安装 OpenJDK 与 Maven

这两个安装包我已经提前准备好：`apache-maven-3.9.16-bin.tar.gz` 与 `openjdk-21_linux-x64_bin.tar.gz`。

```
# 1. 创建安装目录
mkdir -p /usr/local/java
mkdir -p /usr/local/maven

# 2. 解压 JDK 并进行标准化重命名（重命名为 jdk21，方便以后维护）
tar -zxvf openjdk-21_linux-x64_bin.tar.gz -C /usr/local/java/
cd /usr/local/java/ && mv jdk-21* jdk21

# 3. 解压 Maven 并进行标准化重命名（重命名为 maven3）
cd /home/erha
tar -zxvf apache-maven-3.9.16-bin.tar.gz -C /usr/local/maven/
cd /usr/local/maven/ && mv apache-maven-3.9.16* maven3

### 配置环境变量
# 1. 将配置直接追加到配置文件末尾
cat >> /etc/profile << 'EOF'

# Java & Maven Environment Settings
export JAVA_HOME=/usr/local/java/jdk21
export MAVEN_HOME=/usr/local/maven/maven3
export PATH=$JAVA_HOME/bin:$MAVEN_HOME/bin:$PATH
EOF

# 2. 让环境变量立即生效
source /etc/profile

### 配置阿里云镜像和本地仓库
# 1. 创建自定义的本地依赖包存放目录（防止堆在系统盘根目录）
mkdir -p /usr/local/maven/repo

# 2. 覆盖写入精简版的 settings.xml（已包含阿里云公共仓库镜像和本地仓库路径）
cat > /usr/local/maven/maven3/conf/settings.xml << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<settings xmlns="http://maven.apache.org/SETTINGS/1.2.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.2.0
https://maven.apache.org/xsd/settings-1.2.0.xsd">

  <!-- 设置本地依赖包存放路径 -->
  <localRepository>/usr/local/maven/repo</localRepository>

  <pluginGroups></pluginGroups>
  <proxies></proxies>

  <!-- 配置国内阿里云公共仓库加速 -->
```



```

<mirrors>
  <mirror>
    <id>aliyunmaven</id>
    <mirrorOf>*</mirrorOf>
    <name>阿里云公共仓库</name>
    <url>https://maven.aliyun.com/repository/public</url>
  </mirror>
</mirrors>

<profiles></profiles>
</settings>
EOF

```

3. 克隆代码与预配置

登录阿里云镜像仓库（个人版）：

```
docker login --username=erha6657 crpi-ygijwidbw2jwxum2.cn-
hangzhou.personal.cr.aliyuncs.com
```

克隆代码仓库：

```

cd /root
git clone https://gitee.com/erha6657/mall-swarm.git
cd mall-swarm

```

我们的构建脚本和 yaml 目录结构最终将如下所示：

```

mall-swarm-deploy/
├─ build-and-push.sh          # 一键编译+打包Docker+推送阿里云
├─ Dockerfiles/
│   ├─ Dockerfile-gateway
│   ├─ Dockerfile-admin
│   ├─ Dockerfile-auth
│   ├─ Dockerfile-monitor
│   ├─ Dockerfile-portal
│   └─ Dockerfile-search
├─ k8s/
│   ├─ 00-namespace.yaml      # 命名空间 mall
│   ├─ 01-mysql.yaml          # MySQL + PVC + Service
│   ├─ 02-redis.yaml          # Redis + Service
│   ├─ 03-nacos.yaml          # Nacos + Service
│   ├─ 10-mall-gateway.yaml    # 网关
│   ├─ 11-mall-auth.yaml       # 认证中心
│   ├─ 12-mall-admin.yaml      # 后台管理
│   ├─ 13-mall-portal.yaml     # 前台商城
│   ├─ 14-mall-search.yaml     # 搜索服务
│   ├─ 15-mall-monitor.yaml    # 监控中心
│   └─ 20-ingress.yaml        # 统一入口
└─ nacos-config/              # 修改后的Nacos配置

```

4. 替换项目配置 (Localhost -> K8s Service)

在 Kubernetes 内部，服务之间通过 Service 名称 (DNS) 互相访问，而不是 `localhost`。因此，我们需要批量替换 `application-dev.yml` 以及 `config` 目录中的地址：

```
cd /root/mall-swarm

echo "===== 1. 修改 application-dev.yml 中的 Nacos 地址 ====="
for file in $(find . -path "*/src/main/resources/application-dev.yml"); do
    echo "Processing: $file"
    # Nacos 通用格式 localhost:8848 -> nacos-svc:8848
    sed -i 's|localhost:8848|nacos-svc:8848|g' "$file"
    # mall-monitor 特殊格式
    sed -i 's|http://localhost:8848|http://nacos-svc:8848|g' "$file"
done

echo ""
echo "===== 2. 修改 config/ 目录中 Nacos 配置文件 ====="
for file in $(find config -type f -name "*.yaml"); do
    echo "Processing: $file"
    # MySQL 数据库连接
    sed -i 's|jdbc:mysql://localhost:3306|jdbc:mysql://mysql-svc:3306|g' "$file"
    sed -i 's|jdbc:mysql://127.0.0.1:3306|jdbc:mysql://mysql-svc:3306|g' "$file"
    # Redis host
    sed -i 's|host: localhost|host: redis-svc|g' "$file"
    sed -i 's|host: 127.0.0.1|host: redis-svc|g' "$file"
    # MongoDB host
    sed -i 's|localhost:27017|mongodb-svc:27017|g' "$file"
    sed -i 's|host: localhost.*# MongoDB|host: mongodb-svc # MongoDB|g' "$file"
    # RabbitMQ host
    sed -i 's|localhost:5672|rabbitmq-svc:5672|g' "$file"
    sed -i 's|host: localhost.*# RabbitMQ|host: rabbitmq-svc # RabbitMQ|g' "$file"
    # Nacos 地址
    sed -i 's|localhost:8848|nacos-svc:8848|g' "$file"
done

echo ""
echo "===== 3. 验证修改结果 ====="
grep -rn "localhost\|127.0.0.1" config/ || echo " (无，全部替换成功!)"
```

5. 编译后端源码 (跳过 Docker 插件与测试)

```
cd /root/mall-swarm

# 先编译公共模块
mvn clean install -pl mall-common,mall-mbg -DskipTests -Ddocker.skip=true

# 编译所有业务模块 (由于资源有限，不含 mall-search，除非需要)
mvn clean package \
    -pl mall-gateway,mall-admin,mall-auth,mall-monitor,mall-portal \
    -DskipTests \
    -am \
    -Ddocker.skip=true
```

6. 一键生成 Dockerfile 与 K8s YAML

直接复制以下整段到终端，这个脚本会自动帮你生成符合规范的 Dockerfile 以及中间件和微服务的 K8s YAML：

```
cat > /tmp/setup-all.sh << 'SETUPEOF'
#!/bin/bash
set -e

REGISTRY="crpi-ygijwidbw2jwxum2.cn-hangzhou.personal.cr.aliyuncs.com/erha_test1"
DEPLOY_DIR="/root/mall-swarm/deploy"
DOCKER_DIR="${DEPLOY_DIR}/dockerfiles"
K8S_DIR="${DEPLOY_DIR}/k8s"
NAMESPACE="mall"

mkdir -p ${DOCKER_DIR} ${K8S_DIR}

echo "===== 1. 生成 Dockerfile ====="

MODULES=("mall-gateway:8201" "mall-admin:8080" "mall-auth:8401" "mall-monitor:8101" "mall-portal:8085")

for item in "${MODULES[@]"; do
    module="${item%%:*}"
    port="${item##*:*}"
    cat > ${DOCKER_DIR}/Dockerfile-${module} << DOCKERFILE
FROM eclipse-temurin:17-jre-alpine
MAINTAINER erha6657
RUN mkdir -p /app/logs && \
    apk add --no-cache tzdata curl && \
    ln -sf /usr/share/zoneinfo/Asia/Shanghai /etc/localtime && \
    echo "Asia/Shanghai" > /etc/timezone
WORKDIR /app
COPY ${module}/target/*.jar app.jar
EXPOSE ${port}
ENV JAVA_OPTS="-Xms256m -Xmx512m"
ENTRYPOINT ["sh", "-c", "java ${JAVA_OPTS} -jar app.jar"]
DOCKERFILE
    echo " -> ${DOCKER_DIR}/Dockerfile-${module}"
done

echo "===== 2. 生成 K8s YAML ====="

# 命名空间
cat > ${K8S_DIR}/00-namespace.yaml << YAML
apiVersion: v1
kind: Namespace
metadata:
  name: ${NAMESPACE}
YAML

# MySQL
cat > ${K8S_DIR}/01-mysql.yaml << YAML
---
```

```
apiVersion: v1
```

```
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: ${NAMESPACE}
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: alicloud-disk-essd
  resources:
    requests:
      storage: 20Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mysql-svc
  namespace: ${NAMESPACE}
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "root"
            - name: MYSQL_DATABASE
              value: "mall"
          ports:
            - containerPort: 3306
          resources:
            requests: { memory: "512Mi", cpu: "500m" }
            limits: { memory: "1Gi", cpu: "1000m" }
          volumeMounts:
            - name: mysql-data
              mountPath: /var/lib/mysql
```

```

        volumes:
          - name: mysql-data
            persistentVolumeClaim:
              claimName: mysql-pvc
YAML

# Redis
cat > ${K8S_DIR}/02-redis.yaml << YAML
---
apiVersion: v1
kind: Service
metadata:
  name: redis-svc
  namespace: ${NAMESPACE}
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
    type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
          resources:
            requests: { memory: "128Mi", cpu: "100m" }
            limits: { memory: "256Mi", cpu: "500m" }
YAML

# Nacos
cat > ${K8S_DIR}/03-nacos.yaml << YAML
---
apiVersion: v1
kind: Service
metadata:
  name: nacos-svc
  namespace: ${NAMESPACE}

```

```

spec:
  selector:
    app: nacos
  ports:
    - name: http
      port: 8848
      targetPort: 8848
    - name: grpc
      port: 9848
      targetPort: 9848
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nacos
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nacos
  template:
    metadata:
      labels:
        app: nacos
    spec:
      containers:
        - name: nacos
          image: nacos/nacos-server:v2.2.3
          env:
            - name: MODE
              value: "standalone"
          ports:
            - containerPort: 8848
            - containerPort: 9848
          resources:
            requests: { memory: "512Mi", cpu: "500m" }
            limits: { memory: "1Gi", cpu: "1000m" }

```

YAML

MongoDB (轻量, 只给 256Mi)

```
cat > ${K8S_DIR}/04-mongodb.yaml << YAML
```

```

---
apiVersion: v1
kind: Service
metadata:
  name: mongodb-svc
  namespace: ${NAMESPACE}
spec:
  selector:
    app: mongodb
  ports:
    - port: 27017
      targetPort: 27017

```

```

    type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo:7.0
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_DATABASE
              value: "mall"
          resources:
            requests: { memory: "128Mi", cpu: "100m" }
            limits:   { memory: "256Mi", cpu: "500m" }

```

YAML

RabbitMQ (轻量, 只给 256Mi)

```
cat > ${K8S_DIR}/05-rabbitmq.yaml << YAML
```

```

---
apiVersion: v1
kind: Service
metadata:
  name: rabbitmq-svc
  namespace: ${NAMESPACE}
spec:
  selector:
    app: rabbitmq
  ports:
    - port: 5672
      targetPort: 5672
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:

```

```

    app: rabbitmq
  template:
    metadata:
      labels:
        app: rabbitmq
    spec:
      containers:
      - name: rabbitmq
        image: rabbitmq:3.12-management-alpine
        ports:
          - containerPort: 5672
          - containerPort: 15672
        env:
          - name: RABBITMQ_DEFAULT_USER
            value: "guest"
          - name: RABBITMQ_DEFAULT_PASS
            value: "guest"
        resources:
          requests: { memory: "128Mi", cpu: "100m" }
          limits: { memory: "256Mi", cpu: "500m" }

```

YAML

微服务生成函数

```

generate_service() {
  MODULE=$1
  PORT=$2
  cat > ${K8S_DIR}/10-${MODULE}.yaml << YAML
  ---
  apiVersion: v1
  kind: Service
  metadata:
    name: ${MODULE}-svc
    namespace: ${NAMESPACE}
  spec:
    selector:
      app: ${MODULE}
    ports:
      - port: ${PORT}
        targetPort: ${PORT}
    type: ClusterIP
  ---
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: ${MODULE}
    namespace: ${NAMESPACE}
  spec:
    replicas: 1
    selector:
      matchLabels:
        app: ${MODULE}
    template:
      metadata:
        labels:
          app: ${MODULE}

```



```

spec:
  containers:
  - name: ${MODULE}
    image: ${REGISTRY}/${MODULE}:1.0.0
    ports:
    - containerPort: ${PORT}
    env:
    - name: SPRING_PROFILES_ACTIVE
      value: "dev"
    - name: SPRING_CLOUD_NACOS_DISCOVERY_SERVER-ADDR
      value: "nacos-svc:8848"
    - name: SPRING_CLOUD_NACOS_CONFIG_SERVER-ADDR
      value: "nacos-svc:8848"
    resources:
      requests: { memory: "256Mi", cpu: "200m" }
      limits: { memory: "512Mi", cpu: "500m" }
YAML
  echo "  -> ${K8S_DIR}/10-${MODULE}.yaml"
}

generate_service "mall-gateway" "8201"
generate_service "mall-admin" "8080"
generate_service "mall-auth" "8401"
generate_service "mall-monitor" "8101"
generate_service "mall-portal" "8085"

echo "===== 全部生成完毕 ====="
SETUPEOF

# 执行脚本
bash /tmp/setup-all.sh

```

7. 构建后端镜像并推送

在执行推送时，发现网关模块在 Alpine 基础镜像下 `apk add curl` 会报错，我们将 `curl` 从 Dockerfile 移除并重新构建：

```

cd /root/mall-swarm
REGISTRY="crpi-ygijwidbw2jwxum2.cn-hangzhou.personal.cr.aliyuncs.com/erha_test1"

# 构建其他微服务
for module in mall-admin mall-auth mall-monitor mall-portal; do
  echo "Building ${module}..."
  docker build -f deploy/dockerfiles/Dockerfile-${module} -t ${module}:1.0.0 .
  docker tag ${module}:1.0.0 ${REGISTRY}/${module}:1.0.0
  docker push ${REGISTRY}/${module}:1.0.0
  echo "${module} 镜像推送完成!"
done

# 修改并构建网关：去掉 curl
cd /root/mall-swarm/deploy/dockerfiles
sed -i 's/tzdata curl/tzdata/' Dockerfile-mall-gateway

cd /root/mall-swarm
echo "===== 重新构建 mall-gateway ====="

```

```
docker build -f deploy/dockerfiles/Dockerfile-mall-gateway -t mall-gateway:1.0.0
.
docker tag mall-gateway:1.0.0 ${REGISTRY}/mall-gateway:1.0.0
docker push ${REGISTRY}/mall-gateway:1.0.0
```

推送完成后，可以在阿里云镜像仓库中查看到我们的全部后端微服务。

mall-admin	erha_test1	✓ 正常	公开		2026-06-08 21:13:29	2026-06-08 21:13:29
mall-auth	erha_test1	✓ 正常	公开		2026-06-08 21:14:01	2026-06-08 21:14:01
mall-monitor	erha_test1	✓ 正常	公开		2026-06-08 21:14:35	2026-06-08 21:14:35
mall-portal	erha_test1	✓ 正常	公开		2026-06-08 21:15:38	2026-06-08 21:15:38
mall-gateway	erha_test1	✓ 正常	公开		2026-06-08 21:43:34	2026-06-08 21:43:34

三、前端工程：源码获取与容器化构建

这个步骤是在后端基本完成编译后开始进行的。注意：编译前端镜像时，Node.js 请使用 20 及以上的版本。

1. 前端项目地址

项目	Gitee 地址	技术栈	说明
mall-admin-web	https://gitee.com/macrozheng/mall-admin-web	Vue 2.x + Element UI	后台管理系统前端
mall-app-web	https://gitee.com/macrozheng/mall-app-web	Vue 3.x + Vant UI	移动端商城

首先将两个项目克隆到本地服务器：

```
cd /root
git clone https://gitee.com/erha6657/mall-admin-web.git
git clone https://gitee.com/erha6657/mall-app-web.git
```

2. 接口地址代理设计 (解决跨域和 K8s 内网访问)

在开发环境中，API 地址配在 config/dev.env.js。但在 K8s 生产环境中，前端代码在浏览器运行，它必须通过能够连通外网的入口访问后端网关。

我们的最佳实践方案：

直接让 Nginx 托管前端编译好的静态文件，同时在 `nginx.conf` 中配置反向代理规则 `proxy_pass`。遇到发往 `/api/` 的 AJAX 请求，Nginx 自动拦截并在内网转发给 K8s 内部的 `mall-gateway-svc:8201`。前端项目无需修改复杂的打包配置！

3. 一键构建与推送前端镜像脚本

将以下大脚本复制到 Linux 终端中运行。它会**自动生成 Nginx 配置**和**多阶段构建 Dockerfile**，并自动判断编译命令：

```
cat > /root/build-frontend-final.sh << 'SCRIPTEOF'
#!/bin/bash
set -e
REGISTRY="crpi-ygijwidbw2jwxum2.cn-hangzhou.personal.cr.aliyuncs.com/erha_test1"

echo "=====
echo " 1/2 构建 mall-admin-web（后台管理系统）"
echo "=====

cd /root/mall-admin-web

# 创建 nginx.conf
cat > nginx.conf << 'NGX'
server {
    listen 80;
    server_name localhost;
    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://mall-gateway-svc:8201/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
NGX

# 创建 Dockerfile（核心：使用 node:20 避免编译报错）
cat > Dockerfile << 'DKF'
FROM node:20-alpine as builder
WORKDIR /app
COPY . .
RUN npm install --registry=https://registry.npmirror.com --legacy-peer-deps &&
npm run build-only
FROM nginx:stable-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
DKF

echo "开始构建镜像..."
```

```

docker build -t mall-admin-web:1.0.0 .
docker tag mall-admin-web:1.0.0 ${REGISTRY}/mall-admin-web:1.0.0
docker push ${REGISTRY}/mall-admin-web:1.0.0
echo "✅ mall-admin-web 完成!"

echo ""
echo "=====
echo " 2/2 构建 mall-app-web (移动端商城) "
echo "=====

cd /root/mall-app-web

# 自动检测构建命令
FULL_SCRIPTS=$(cat package.json)
BUILD_CMD=""
OUTPUT_DIR="/app/dist"

if echo "$FULL_SCRIPTS" | grep -q '"build:h5"'; then
    BUILD_CMD="build:h5"
    OUTPUT_DIR="/app/dist/build/h5"
elif echo "$FULL_SCRIPTS" | grep -q '"build-only"'; then
    BUILD_CMD="build-only"
elif echo "$FULL_SCRIPTS" | grep -q '"build"'; then
    BUILD_CMD="build"
else
    echo "❌ 找不到构建命令"
    grep '"scripts"' -A 20 package.json
    exit 1
fi
echo "构建命令: npm run ${BUILD_CMD}"

cat > nginx.conf << 'NGX'
server {
    listen 80;
    server_name localhost;
    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://mall-gateway-svc:8201/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
NGX

cat > Dockerfile << DKF
FROM node:20-alpine as builder
WORKDIR /app
COPY . .
RUN npm install --registry=https://registry.npmmirror.com --legacy-peer-deps &&
    npm run ${BUILD_CMD}
FROM nginx:stable-alpine

```

```

COPY --from=builder ${OUTPUT_DIR} /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
DKF

echo "开始构建镜像..."
docker build -t mall-app-web:1.0.0 .
docker tag mall-app-web:1.0.0 ${REGISTRY}/mall-app-web:1.0.0
docker push ${REGISTRY}/mall-app-web:1.0.0
echo "✅ mall-app-web 完成!"

echo ""
echo "===== "
echo " 📦 所有镜像已推送完毕! "
echo "===== "
SCRIPTEOF

# 执行构建并推送
chmod +x /root/build-frontend-final.sh
bash /root/build-frontend-final.sh

```

推送完成后，可以在阿里云镜像控制台看到前端镜像已就绪。

mall-admin-web	erha_test1	✅ 正常	公开 🚩	⋮	2026-06-08 22:05:16	2026-06-08 22:05:16
mall-app-web	erha_test1	✅ 正常	公开 🚩	⋮	2026-06-08 22:06:05	2026-06-08 22:06:05

4. 生成前端 K8s 部署 YAML

为了保持目录的整洁，我们将前端应用的 K8s yaml 文件追加到之前的 `/root/mall-swarm/deploy/k8s` 目录中：

```

REGISTRY="crpi-ygijwidbw2jwxum2.cn-hangzhou.personal.cr.aliyuncs.com/erha_test1"
K8S_DIR="/root/mall-swarm/deploy/k8s"
NAMESPACE="mall"

# --- mall-admin-web ---
cat > ${K8S_DIR}/21-mall-admin-web.yaml << YAML
---
apiVersion: v1
kind: Service
metadata:
  name: mall-admin-web-svc
  namespace: ${NAMESPACE}
spec:
  selector:
    app: mall-admin-web
  ports:
    - port: 80

```

```

        targetPort: 80
        type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mall-admin-web
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mall-admin-web
  template:
    metadata:
      labels:
        app: mall-admin-web
    spec:
      containers:
        - name: admin-web
          image: ${REGISTRY}/mall-admin-web:1.0.0
          ports:
            - containerPort: 80
          resources:
            requests: { memory: "64Mi", cpu: "50m" }
            limits:   { memory: "128Mi", cpu: "200m" }

```

YAML

```

# --- mall-app-web ---
cat > ${K8S_DIR}/22-mall-app-web.yaml << YAML

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: mall-app-web-svc
  namespace: ${NAMESPACE}
spec:
  selector:
    app: mall-app-web
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mall-app-web
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mall-app-web
  template:

```

```

metadata:
  labels:
    app: mall-app-web
spec:
  containers:
  - name: app-web
    image: ${REGISTRY}/mall-app-web:1.0.0
    ports:
    - containerPort: 80
  resources:
    requests: { memory: "64Mi", cpu: "50m" }
    limits: { memory: "128Mi", cpu: "200m" }

```

YAML

```

# --- Ingress 统一入口 (可选) ---
cat > ${K8S_DIR}/30-ingress.yaml << YAML
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mall-ingress
  namespace: ${NAMESPACE}
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /admin
        pathType: Prefix
        backend:
          service:
            name: mall-admin-web-svc
            port:
              number: 80
      - path: /app
        pathType: Prefix
        backend:
          service:
            name: mall-app-web-svc
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: mall-gateway-svc
            port:
              number: 8201

```

YAML

```
echo "✅ K8s 前端部署文件已生成!"
```

四、阿里云 ACK 集群配置与全栈部署

1. ACK 集群准备

在阿里云控制台创建 Kubernetes 集群：

- **集群类型**：ACK 托管基础版，网络插件 Flannel。
- **Worker 节点**：1台 `ecs.c7a.xlarge` (8核16G 抢占式实例，挂载 60G ESSD 云盘数据盘)。
- **连接集群**：在控制台复制 `kubeconfig` 内容，并在本地服务器配置到 `~/.kube/config` 中以连通集群。

所选配置	产品类别	产品配置	数量	付费方式	购买周期	费用
生成 OpenAPI 请求参数 AI 配置优化建议	ACK 集群	地域：华东1（杭州） Kubernetes 版本：1.36.1-aliyun.1 容器运行时：containerd 2.1.6 容器网络：172.16.0.0/12 服务网络：192.168.0.0/16	1	无	无	
	集群系统组件	网络插件：flannel 存储插件：managed-csi-provisioner 日志服务：loongcollector ack-node-problem-detector 容器监控：metrics-server 服务发现：coredns		无	无	
	云服务器 - Worker	系统盘：ESSD 云盘 - 40G 数据盘：ESSD 云盘 - 60G	1	抢占式实例	无	
	负载均衡(CLB) - API Server		1	按量付费 - 按使用量计费	无	¥ 0.049 /LCU ¥ 0.147 /时
	弹性网卡	根据集群配置不同，将会创建 1-2 块弹性网卡	1-2	无	无	
	资源编排	将自动创建名称以 k8s-for-cs 开头的资源栈	1	无	无	
	弹性伸缩 (Auto Scaling)	将使用弹性伸缩组创建 worker 节点	1	无	无	
	安全组		1	无	无	
	访问控制		1	无	无	
	弹性公网 IP (EIP)	公网带宽：按使用流量计费	1	按量付费	无	¥ 0.800 /GB ¥ 0.020 /时

2. 核心 YAML 整合与终极修复 (all-in-one)

由于在分体式部署中遇到了一些典型坑（例如 NodePort 公网不通、Nacos 掉线后 Redis 回退连本地报错、Logstash 连不上卡死等，详见后文总结），我们最终将所有配置整合为了一个 `all-in-one.yaml`，并加入了**强效修复机制**。

主要改进点：

1. **环境变量兜底**：强制注入 `SPRING_DATA_REDIS_HOST=redis-svc`，防止 Nacos 配置丢失导致微服务崩溃。
2. **禁用 Logstash**：利用 K8s ConfigMap 挂载自定义的 `logback-spring.xml`（只保留控制台输出），彻底解决因 Logstash 连不上而导致启动卡死的问题。
3. **前端 LoadBalancer**：将原来前端的 ClusterIP/NodePort 改为 `LoadBalancer`，利用阿里云自动生成 SLB 提供公网入口。
4. **内网拉取加速**：所有的镜像地址全部替换为了 `-vpc` 后缀，走阿里云内网拉取，速度快且免公网流量费。

执行以下脚本生成终极部署文件：

```
mkdir -p /root/mall-swarm/deploy/k8s-final
cat > /root/mall-swarm/deploy/k8s-final/all-in-one.yaml << 'YAMLEOF'
# =====
# mall-swarm 完整部署 YAML（最终可用版）
# 镜像仓库：阿里云杭州 VPC 内网
# 暴露方式：LoadBalancer（自动分配公网 IP）
# 修复：Redis localhost、Logstash 禁用、Nacos 配置兜底
# =====

apiVersion: v1
```



```

kind: Namespace
metadata:
  name: mall

---
# ===== Logback ConfigMap (禁用 Logstash) =====
apiVersion: v1
kind: ConfigMap
metadata:
  name: logback-config
  namespace: mall
data:
  logback-spring.xml: |
    <?xml version="1.0" encoding="UTF-8"?>
    <configuration>
      <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
          <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level
%logger{36} - %msg%n</pattern>
        </encoder>
      </appender>
      <root level="INFO">
        <appender-ref ref="CONSOLE" />
      </root>
      <logger name="org.springframework" level="INFO" />
      <logger name="com.macro.mall" level="DEBUG" />
    </configuration>

---
# ===== 前端 Nginx ConfigMap (后台与移动端) =====
apiVersion: v1
kind: ConfigMap
metadata:
  name: admin-web-nginx-config
  namespace: mall
data:
  default.conf: |
    server {
      listen 80;
      location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
      }
      location /api/ {
        proxy_pass http://mall-admin-svc:8080/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
      }
    }

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-web-nginx-config

```

```

    namespace: mall
data:
  default.conf: |
    server {
      listen 80;
      location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
      }
      location /api/ {
        proxy_pass http://mall-portal-svc:8085/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
      }
    }

---
# ===== 中间件: MySQL =====
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: mall
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: alicloud-disk-essd
  resources:
    requests:
      storage: 20Gi

---
apiVersion: v1
kind: Service
metadata:
  name: mysql-svc
  namespace: mall
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
  type: ClusterIP

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: mall
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:

```

```

    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: crpi-ygijwidbw2jwxum2-vpc.cn-hangzhou.personal.cr.aliyuncs.com/erha_test1/mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "root"
            - name: MYSQL_DATABASE
              value: "mall"
          ports:
            - containerPort: 3306
          resources:
            requests: { memory: "512Mi", cpu: "500m" }
            limits: { memory: "1Gi", cpu: "1000m" }
          volumeMounts:
            - name: mysql-data
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-data
          persistentVolumeClaim:
            claimName: mysql-pvc
---
# ===== 中间件: Redis / Nacos / MongoDB / RabbitMQ
=====
# (此处省略中间件 YAML, 与前文一致, 只需注意镜像地址换为 -vpc 结尾的内网地址即可)
# ...
---
apiVersion: v1
kind: Service
metadata:
  name: redis-svc
  namespace: mall
spec:
  selector: { app: redis }
  ports: [ { port: 6379, targetPort: 6379 } ]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: mall
spec:
  replicas: 1
  selector: { matchLabels: { app: redis } }
  template:
    metadata: { labels: { app: redis } }
    spec:
      containers:
        - name: redis

```

```

        image: crpi-ygijwidbw2jwxum2-vpc.cn-
hangzhou.personal.cr.aliyuncs.com/erha_test1/redis:7-alpine
        ports: [ { containerPort: 6379 } ]
        resources:
            requests: { memory: "128Mi", cpu: "100m" }
            limits:   { memory: "256Mi", cpu: "500m" }

---
# ===== 微服务: mall-gateway =====
apiVersion: v1
kind: Service
metadata:
  name: mall-gateway-svc
  namespace: mall
spec:
  selector:
    app: mall-gateway
  ports:
    - port: 8201
      targetPort: 8201
  type: ClusterIP

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mall-gateway
  namespace: mall
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mall-gateway
  template:
    metadata:
      labels:
        app: mall-gateway
    spec:
      containers:
        - name: mall-gateway
          image: crpi-ygijwidbw2jwxum2-vpc.cn-
hangzhou.personal.cr.aliyuncs.com/erha_test1/mall-gateway:1.0.0
          ports:
            - containerPort: 8201
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "dev"
            - name: SPRING_CLOUD_NACOS_DISCOVERY_SERVER-ADDR
              value: "nacos-svc:8848"
            - name: SPRING_CLOUD_NACOS_CONFIG_SERVER-ADDR
              value: "nacos-svc:8848"
            - name: SPRING_DATA_REDIS_HOST
              value: "redis-svc"
            - name: SPRING_DATA_REDIS_PORT
              value: "6379"
            - name: LOGGING_CONFIG

```

```

        value: "/app/conf/logback-spring.xml"
      resources:
        requests: { memory: "384Mi", cpu: "300m" }
        limits:   { memory: "768Mi", cpu: "800m" }
      volumeMounts:
        - name: logback-config
          mountPath: /app/conf
      volumes:
        - name: logback-config
          configMap:
            name: logback-config

# (其他 mall-auth, mall-admin, mall-monitor, mall-portal 类似配置)
# ...

---
# ===== 前端: mall-admin-web (LoadBalancer) =====
apiVersion: v1
kind: Service
metadata:
  name: mall-admin-web-svc
  namespace: mall
spec:
  selector:
    app: mall-admin-web
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mall-admin-web
  namespace: mall
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mall-admin-web
  template:
    metadata:
      labels:
        app: mall-admin-web
    spec:
      containers:
        - name: admin-web
          image: crpi-ygijwidbw2jwxum2-vpc.cn-hangzhou.personal.cr.aliyuncs.com/erha_test1/mall-admin-web:1.0.0
          ports:
            - containerPort: 80
          volumeMounts:
            - name: nginx-config
              mountPath: /etc/nginx/conf.d/default.conf
              subPath: default.conf

```

```

    resources:
      requests: { memory: "64Mi", cpu: "50m" }
      limits:   { memory: "128Mi", cpu: "200m" }
    volumes:
      - name: nginx-config
        configMap:
          name: admin-web-nginx-config

---
# ===== 前端: mall-app-web (LoadBalancer) =====
apiVersion: v1
kind: Service
metadata:
  name: mall-app-web-svc
  namespace: mall
spec:
  selector:
    app: mall-app-web
  ports:
    - port: 80
      targetPort: 80
    type: LoadBalancer

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mall-app-web
  namespace: mall
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mall-app-web
  template:
    metadata:
      labels:
        app: mall-app-web
    spec:
      containers:
        - name: app-web
          image: crpi-ygijwidbw2jwxum2-vpc.cn-
hangzhou.personal.cr.aliyuncs.com/erha_test1/mall-app-web:1.0.0
          ports:
            - containerPort: 80
          volumeMounts:
            - name: nginx-config
              mountPath: /etc/nginx/conf.d/default.conf
              subPath: default.conf
          resources:
            requests: { memory: "64Mi", cpu: "50m" }
            limits:   { memory: "128Mi", cpu: "200m" }
      volumes:
        - name: nginx-config
          configMap:
            name: app-web-nginx-config

```

3. 执行部署并持续监控

清空旧的部署（如果存在），然后一键 Apply:

```
cd /root/mall-swarm/deploy/k8s-final

echo "===== 1. 删除旧的微服务 Pod（让它们重新拉镜像） ====="
kubectl -n mall delete deploy --all --ignore-not-found

echo "===== 2. 重新 apply ====="
kubectl apply -f all-in-one.yaml

echo "===== 3. 持续监控 Pod 状态 ====="
kubectl -n mall get pods -w
```

等待所有组件启动成功（Running），我们可以在 K8s 仪表盘或者命令行看到：

```
[root@localhost k8s]# kubectl -n mall get pods
NAME                                READY   STATUS    RESTARTS   AGE
mall-admin-69968c5865-fkxbh         1/1     Running   0           101s
mall-admin-web-55cf5569b5-h4s92     1/1     Running   0           23m
mall-app-web-845f5b9b89-cmc6z       1/1     Running   0           23m
mall-auth-5674f8d98b-5rxrd          1/1     Running   3 (19m ago) 23m
mall-gateway-77b6fbc7c4-6j49w       1/1     Running   3 (18m ago) 23m
mall-monitor-8585d7bc5c-rg8sb       1/1     Running   4 (18m ago) 23m
mall-portal-f764bdf85-d5b6b         1/1     Running   0           101s
mongodb-554c8674db-ssqcd            1/1     Running   0           19m
mysql-76c89965f-26449               1/1     Running   0           19m
nacos-54467f6497-fbkmq              1/1     Running   0           19m
rabbitmq-fcd5df5cb-x8pb6            1/1     Running   0           19m
redis-7f54c4b6d6-srnv5              1/1     Running   0           19m
```

获取我们的前端公网访问地址：

```
kubectl -n mall get svc | grep LoadBalancer

# 输出示例：
# mall-admin-web-svc    LoadBalancer    192.168.6.250    47.97.173.142
80:30080/TCP    97m
# mall-app-web-svc      LoadBalancer    192.168.121.158  47.97.194.118
80:30081/TCP    97m
```

五、部署后初始化与验证

1. 补装 MinIO 对象存储

系统需要 MinIO 存储图片。我们在 K8s 中补充部署 MinIO 并配置公开读写权限：

```
REGISTRY="crpi-ygijwidbw2jwxum2-vpc.cn-
hangzhou.personal.cr.aliyuncs.com/erha_test1"

# 1. 部署 MinIO (YAML 略，标准 Deployment 暴露 9000 和 9001 端口)
# ...
```

```
# 2. 等待启动后, 通过 port-forward 配置 bucket
kubectl -n mall port-forward svc/minio-svc 9000:9000 &
sleep 2

# 使用 mc 客户端设置 bucket 并公开访问
mc alias set myminio http://localhost:9000 minioadmin minioadmin
mc mb myminio/mall --ignore-existing
mc anonymous set public myminio/mall

# 验证 bucket
mc ls myminio/
kill %1 2>/dev/null
```

2. Nacos 配置文件推送

因为我们选用的是 Nacos Standalone 模式（内嵌 Derby 数据库不持久化），每次新建 Pod 都需要重新把商城配置推送进去：

```
# 获取 Nacos Token
LOGIN_RESP=$(curl -s -X POST "http://$(kubectl -n mall get svc nacos-svc -o
jsonpath='{.spec.clusterIP}'):8848/nacos/v1/auth/login" \
-d "username=nacos&password=nacos")
TOKEN=$(echo "$LOGIN_RESP" | grep -o '"accessToken":"[^"]*"' | cut -d'"' -f4)

# 修改本地配置文件指向 MinIO K8s 服务
cd /root/mall-swarm
sed -i 's|http://localhost:9000|http://minio-svc:9000|g' config/admin/mall-
admin-dev.yaml
sed -i 's|http://localhost:9000|http://minio-svc:9000|g' config/portal/mall-
portal-dev.yaml

# 重新推送配置到 Nacos
CONTENT=$(cat config/admin/mall-admin-dev.yaml | base64 -w0)
curl -s -X POST "http://$(kubectl -n mall get svc nacos-svc -o
jsonpath='{.spec.clusterIP}'):8848/nacos/v1/cs/configs?accessToken=$TOKEN" \
-d "dataId=mall-admin-dev.yaml&group=DEFAULT_GROUP&content=$CONTENT&type=yaml"

CONTENT=$(cat config/portal/mall-portal-dev.yaml | base64 -w0)
curl -s -X POST "http://$(kubectl -n mall get svc nacos-svc -o
jsonpath='{.spec.clusterIP}'):8848/nacos/v1/cs/configs?accessToken=$TOKEN" \
-d "dataId=mall-portal-
dev.yaml&group=DEFAULT_GROUP&content=$CONTENT&type=yaml"

# 重启 admin 和 portal（让新配置生效）
kubectl -n mall rollout restart deploy mall-admin mall-portal
```

3. 数据库初始化

进入 MySQL 容器导入初始化的 SQL 数据：

```
kubectl -n mall exec -i deploy/mysql -- mysql -uroot -proot mall < /root/mall-
swarm/document/sql/mall.sql
```

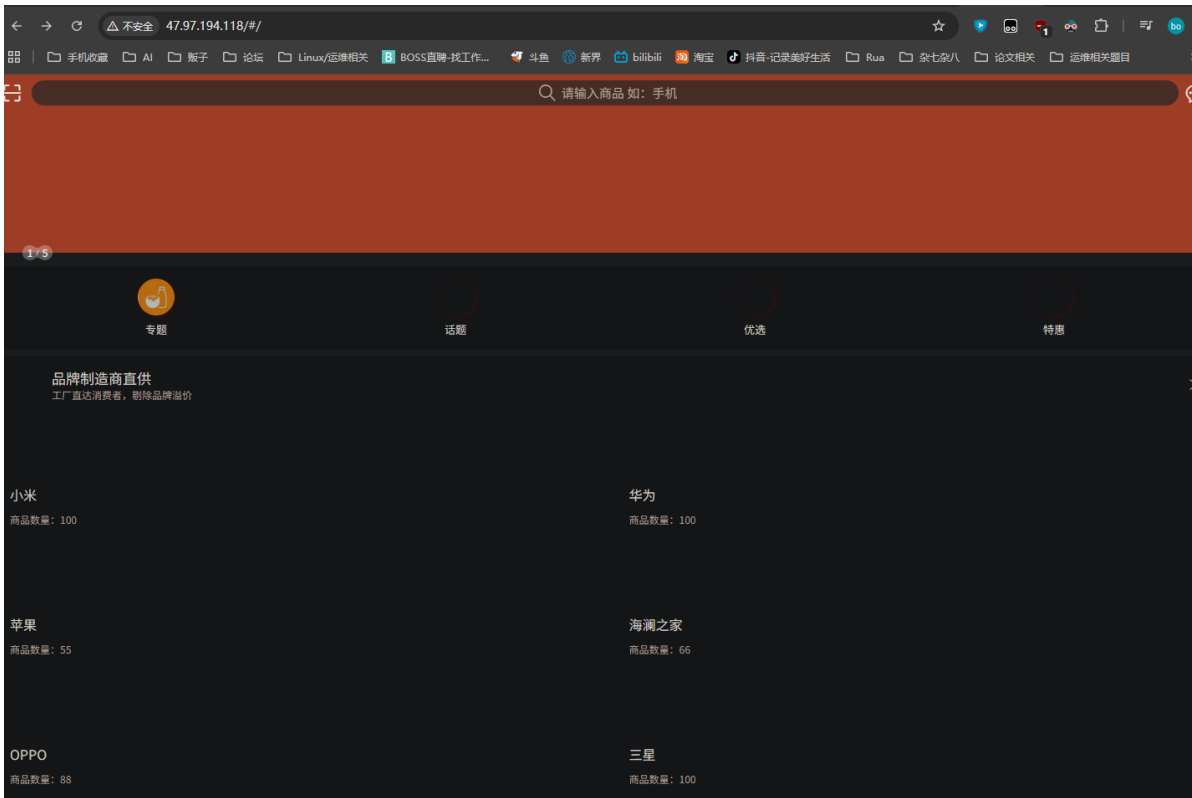

4. 效果展示

配置全部完毕！通过 LoadBalancer 的公网 IP 访问页面：

后台商品列表（搜索功能正常工作）：



移动端前台展示：



注：关于前端图片丢失的说明

由于原项目初始化 SQL 中的图片 URL 指向原作者的 OSS (`macro-oss.oss-cn-shenzhen.aliyuncs.com`)，该桶开启了防盗链或链接失效，导致页面出现图片缺失。这不影响业务逻辑测试，如需完美展示可以手动上传新图片到我们自己搭建的 MinIO 中并替换数据库记录。

✖ 4. Nacos 配置丢失导致服务瘫痪

- **现象：**报错 `config[dataId=mall-admin-dev.yaml, group=DEFAULT_GROUP] is empty`。
- **原因：**Nacos 采用了 standalone 模式，内置 Derby 数据库无持久化。一旦 Nacos Pod 重启，所有配置荡然无存。
- **解决：**关键配置用环境变量兜底；部署后通过写好的 curl 脚本一键将所有 config 重新推入 Nacos 中。

✖ 5. MySQL 连接报 Public Key Retrieval is not allowed

- **现象：**微服务连不上数据库，JDBC 报错。
- **原因：**MySQL 8.0 默认使用了更加安全的 `caching_sha2_password` 认证插件。
- **解决：**修改 Nacos 里的数据库连接串，追加参数 `allowPublicKeyRetrieval=true&useSSL=false`。

✖ 6. 前端 Nginx 代理 502 Bad Gateway

- **现象：**浏览器访问前端，API 请求报错 502。
- **原因：**网关 Pod (`mall-gateway-svc:8201`) 尚未就绪，或前端 Nginx 配置中的域名写错。
- **解决：**确保 Nginx `proxy_pass` 指向正确的 K8s DNS 且结尾包含 `/`，并等待后端网关变为 Running 状态。

✖ 7. NodePort 外部无法访问 (端口 000)

- **现象：**用 K8s NodePort 暴露服务后，公网访问超时，甚至在节点本地 `curl 127.0.0.1:30080` 也不通。
- **原因：**阿里云 ACK 中 Flannel 网络下的 kube-proxy 规则有时会存在限制，或者节点安全组未打通。
- **解决：**直接弃用 NodePort，使用云厂商标准的 **LoadBalancer** 类型的 Service。阿里云 CCM 会自动帮你配置和购买一个 SLB 实例代理流量。

最终资源核算 (8核16G)

MySQL:	1.00 Gi
Redis:	0.25 Gi
Nacos:	1.00 Gi
MongoDB:	0.25 Gi
RabbitMQ:	0.25 Gi
mall-gateway:	0.50 Gi
mall-admin:	0.50 Gi
mall-auth:	0.50 Gi
mall-monitor:	0.50 Gi
mall-portal:	0.50 Gi
admin-web:	0.13 Gi
app-web:	0.13 Gi

总计约: 5.51 Gi  (16G 内存绰绰有余)

